

What is JavaScript?

So far, we have discussed HTML and CSS, and now, we know how to create a web page and give styles to it. But still, everything is just a presentation. For example, we can create a button using the `<button>` tag and we can also apply any sort of CSS to it. But, nothing happens when the button is clicked. This is static and today, **websites are dynamic**.

Things change on a website without reloading the page. **JavaScript is used to make these HTML elements dynamic**.

JavaScript is a vast programming language. It has too many concepts. From the beginner level to the advanced level, there is so much in JavaScript. Then, we have node.js and various frameworks and libraries. As mentioned, it is vast and huge. But we don't need to go this far because this is an introduction.

The main focus of this section is to explain **how to use JavaScript with HTML and CSS**.

We will start from the basics of JavaScript such as declaring variables and using functions. Then, we will move to DOM and how to use JavaScript with HTML and CSS. But before moving further, let's discuss JavaScript and its history briefly.

What is JavaScript?

By definition, **JavaScript is a high-level, multi-paradigm programming language**. It is one of the core technologies of the World Wide Web(WWW), the other two being HTML and CSS. JavaScript is a crucial and essential part of web application development. HTML creates a basic structure, and CSS transforms it into an attractive version. A website with only HTML and CSS is a static website that is of almost no use. But a web application is not about structure and presentation, right? The main focus of a website is user interaction. Suppose, there is a beautiful button on a web page, but if nothing happens when clicked, it is useless. You can make a button work by applying JavaScript on it. If you need a dynamic website, you need to use JavaScript with HTML and CSS.

History of JavaScript

In the early to mid-1990s, the web was young and everything was static. HTML was the only web at that time. People soon realized that the web cannot be static only, it needs to be dynamic and there should be features like animations, and interactions. To make the web dynamic, Dom manipulation was required, and for it, a scripting language was needed. But at that time, Java was emerging in the web community. So the developers decided, why not create a simple scripting language for designing while the heavy tasks were left for Java.

Brendan Eich created JavaScript in the year 1996 when he was working at Netscape communication. JavaScript was created in only ten days. It was not supposed to be what is today.

Summary

- JavaScript is one of the core technologies of the World Wide Web(WWW).
- It is used to create dynamic websites.
- It was created by Brendan Eich in 1996.

Basics of Javascript

JavaScript is a vast programming language that covers several concepts and most importantly, many of these concepts in different ways. We are not learning JavaScript in this course, but it is necessary to discuss some of the basics, such as variables, and functions.

Before starting Javascript with HTML and CSS, let's go through some of the basics of Javascript programming language.

Variables, data types, and operators

Variables

Let's start with one of the most basic terms used in programming - variable.

A variable is defined as **a container that is used to hold values.**

```
1  var x;  
2  var y;  
3  var z;
```

x, y, and z are three variables that are **declared using the "var" keyword.** Two other keywords can also be used to declare variables - **let and const.**

As of now, these variables do not hold any values. To initialize them, **We use the assignment operator(=).**

```
1  var x = 1;  
2  var y = 2;  
3  var z = 3;
```

Declaring a variable with values is called initializing. We can also change these values later by using the assignment operator.

```
1  var x = 1;  
2  var y = 2;  
3  var z = 3;  
4  
5  x = 100;
```

Data types

A variable holds data. Several types of data can be stored in a variable. In the above example, All the variables were holding numerical values. We can also store **single-precision numbers, double-precision numbers, strings, boolean values, objects, date**, etc, in a variable. These are different data types used in the programming world. A data type indicates the characteristics of the data stored in a variable.

In the other programming languages such as Java, it is mandatory to specify the data type of a variable while declaring, and moreover, a variable can hold only that type of data. But there are no such restrictions in Javascript.

Data types in Javascript are divided into two categories - **Primitive and Non-primitive**.

Following is the list of Primitive data types:

- Number
- String
- Boolean
- Undefined

```
1  var num = 100;
2  var str = "Hello World";
3  var bool = true;
4  var und = undefined;
```

Following is the list of non-primitive data types:

- Object
- Array
- Date

```
1  var obj = {
2    |    str : "Hello World"
3  }
4
5  var date = new Date();
6
7  var arr = [1,2,3,4,5]
```

If you have knowledge of any popular programming language, you will easily understand all these data types.

Functions

While coding, we may face situations when we need to execute the same code at different times. For example, we need to find the sum of two numbers five times in a program. The numbers are different every time. we need to write the same logic for five times throughout the program. It will work fine, but do we think it is efficient? The answer is no. It is not efficient because writing the same part again and again, not only increases the length of the code but also the time and effort. Here enters the concept of functions.

Functions are called the main building blocks of a program. They provide **code reusability and helps reducing time and effort.**

For the above example, we can create a function, that accepts two values and returns their sum. we can call this function anytime anywhere we want. Thus removing the need for repetition.

Functions are one of the most important parts of programming. Almost every language has the concept of functions. In this chapter, we will discuss all about functions in JavaScript.

Declaring a function

To create a function in JavaScript, we need **the 'function' keyword** . Observe the following JavaScript code.

```
1  function demoFunction(){
2
3      console.log("This is a function");
4
5  }
```

A function in JavaScript is declared using the 'function' keyword followed by the name of the function and two parentheses. In the above code, the name of the function is 'demoFunction'. we can also see parenthesis attached to it. These parentheses can be empty or can have parameters. We will discuss the parameters later in the lesson. Currently, there are no parameters for this function.

The body of the function is written inside the curly brackets, similar to if and switch statements.

Calling a function

Declaring a function is not enough. **Nothing will happen until the function is called.**

To call a function, simply write the name of the function with the parenthesis.

```
1  function demoFunction(){
2
3      console.log("This is a function")
4
5  }
6
7  demoFunction();
```

Let's execute the above code and check the output.

```
PS E:\javascript> node example.js
This is a function
PS E:\javascript> █
```

The function is executed once because I called it only a single time.

The main objective of using functions is to avoid code repetition and offer better code reusability. Let me show how this is possible with functions.

```
1  function demoFunction(){
2
3      console.log("This is a function")
4
5  }
6
7
8  demoFunction();
9  demoFunction();
10 demoFunction();
```

The demoFunction() is called three times.

```
PS E:\javascript> node example.js
This is a function
This is a function
This is a function
PS E:\javascript> █
```

The console statement inside the demoFunction() is executed three times while actually, it was written once. This is the benefit of using functions. Create a function and call it anytime and anywhere we want.

Parameters

we can always pass data to a function. **The data is passed as parameters** or also known as arguments. Let's go to the example we were discussing earlier in this lesson.

```
1  function add(a,b){
2
3      console.log("Sum of a and b is: ", a+b);
4  }
```

Observe the add() function. we can see, it has two parameters - a and b, which are then used inside the function body. Similarly, we need to pass the values of these parameters while calling the function.

```
1  function add(a,b){
2
3      console.log("Sum of a and b is: ", a+b);
4  }
5
6  add(10,20);
```

The value of a is 10 and b is 20. Let's check the output.

```
PS E:\javascript> node example.js
Sum of a and b is: 30
PS E:\javascript> █
```

we can also pass the variables to a function as parameters.

```
1  function add(a,b){
2
3      console.log("Sum of a and b is: ", a+b);
4  }
5
6  var a = 10;
7  var b = 20;
8
9  add(a,b);
```

But remember, **the names of parameters in the function declaration have no relation with the name of the variables passed as the parameters while**

function calling.

```
1  function add(a,b){
2
3      console.log("Sum of a and b is: ", a+b);
4  }
5
6  var a = 10;
7  var b = 20;
8
9  add(a,b);
10
11 add(b,a);
```

- During the first function call - add(a,b), the value of a in the add() function is 10 and b is 20.
- During the second function call - add(b,a), the value of a in the add() function is 20 and b is 10.

```
PS E:\javascript> node example.js
Sum of a and b is: 30
Sum of a and b is: 30
PS E:\javascript> █
```

Summary

- A variable can be declared using var, let, or const keywords.
- To assign a value to a variable, use the assignment operator.
- There are two categories of data types in JavaScript - Primitive and Non-primitive.
- The primitive data types consist of Number, String, Boolean, and Undefined.
- The Non-primitive data types consist of Object, Date, and Array.
- Functions are the code of blocks that are used for code reusability.
- A function is created using the function keyword and it can be called any number of times using the function name.
- The values passed to a function are called parameters.

DOM

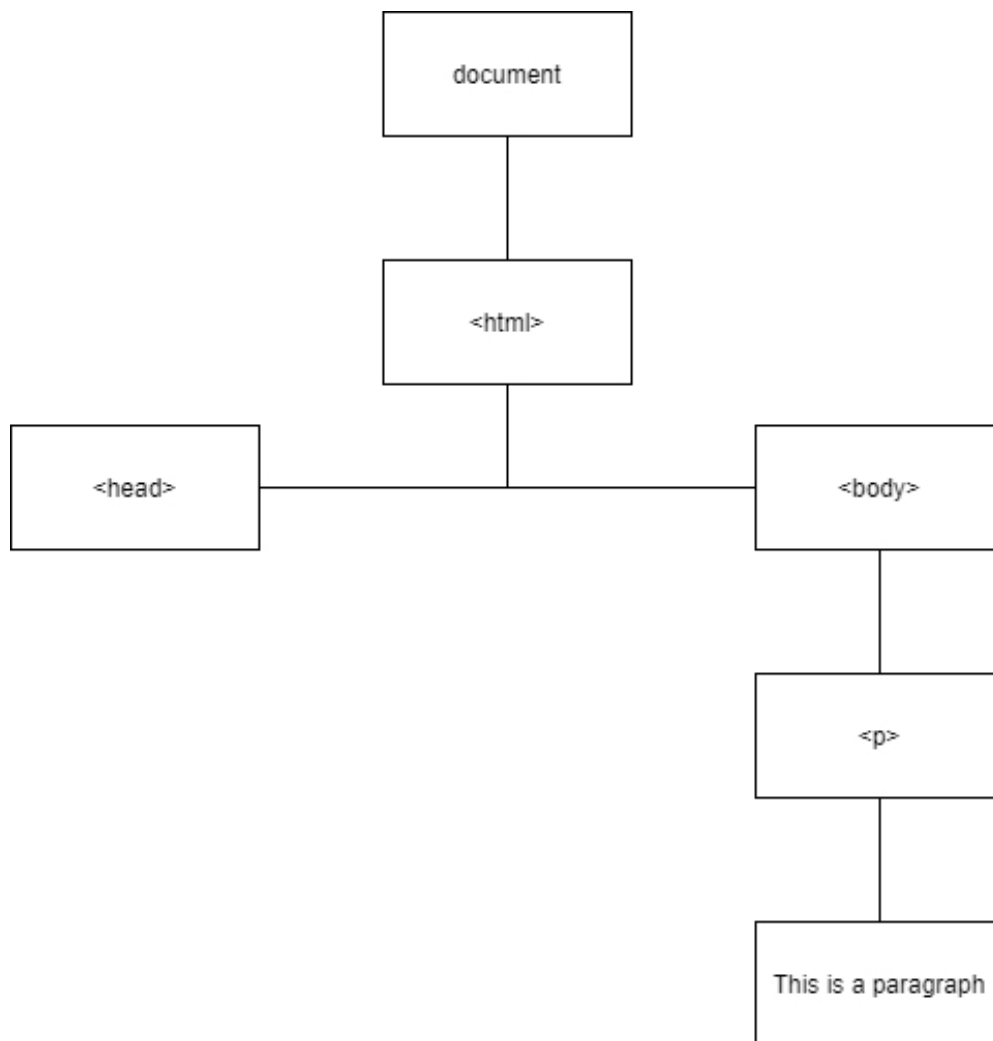
DOM stands for **Document Object Model** . It is a very important concept on the road to becoming a web developer. So in this chapter, we will discuss what is DOM and how it is used with JavaScript.

What is DOM?

When the HTML file is loaded into a browser, **a tree-like structure** is created. This structure has **various nodes** , and these **nodes represent various elements of the HTML document**.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6          |
7          <p>
8              |   This is a paragraph.
9              </p>
10         |
11         </body>
12    </html>
```

The DOM structure of the HTML document will look like the following structure.



The structure starts from `<html>` tag, that is also the root element. Then, `<head>` and `<body>` are two different nodes. In the body, there is a `<p>` tag which is a node of the `<body>` tag.

The tree structure will grow as the more tags are added in the HTML document.

Use of DOM

So far, we created HTML documents using various tags and then applied CSS. Everything is still static. Once loaded in the browser, nothing changes. But websites are not like that.

Suppose, there is a button and when clicked on that button, something happens. Say, the button click changes the text of a `<p>` tag. This is done by **manipulating DOM with the help of JavaScript**. In simple words, JavaScript is used to create dynamic HTML by making changes in the DOM tree.

Take the above example. To change the content of the <p> tag, we will use JavaScript to access that node, and then by using various DOM methods, it is manipulated accordingly.

With JavaScript, we can manipulate almost everything in HTML such as content, styles, even add new elements and remove the existing ones.

Summary

- DOM is a tree-like structure that is created when an HTML document is loaded in a browser.
- Every node of a DOM tree represents an HTML element.
- DOM can be manipulated to make dynamic changes in an HTML document.

HTML events and JavaScript

HTML events

HTML events are **attributes** that are used to make something happen. For example, a button click popping a message. Another example is, popping a message when page loads or when the input changes.

HTML events are very important because they are used to **convert the static HTML elements into dynamic**. One of the most important uses of these events is that JavaScript functions can be triggered using them. Although a very basic DOM manipulation can be done using HTML, serious manipulation is done using functions. This is why HTML events are very important.

There are several HTML events. They are divided into different categories:

- Keyboard events
- Mouse events
- Drag events
- Form events
- Windows events
- Media events
- Clipboard events

Further, there are several events in each of these categories. The main focus of this chapter is to understand, how to use these events to trigger something.

Triggering alert()

The alert() function is pop up that appears on the screen with a message. Let's see how can we trigger a pop up using the onclick mouse event.

Mouse events are one of the most commonly used HTML events. These events are triggered when the user does something with the mouse. For example, clicking on something, double-clicking on something, hovering over something, and many more.

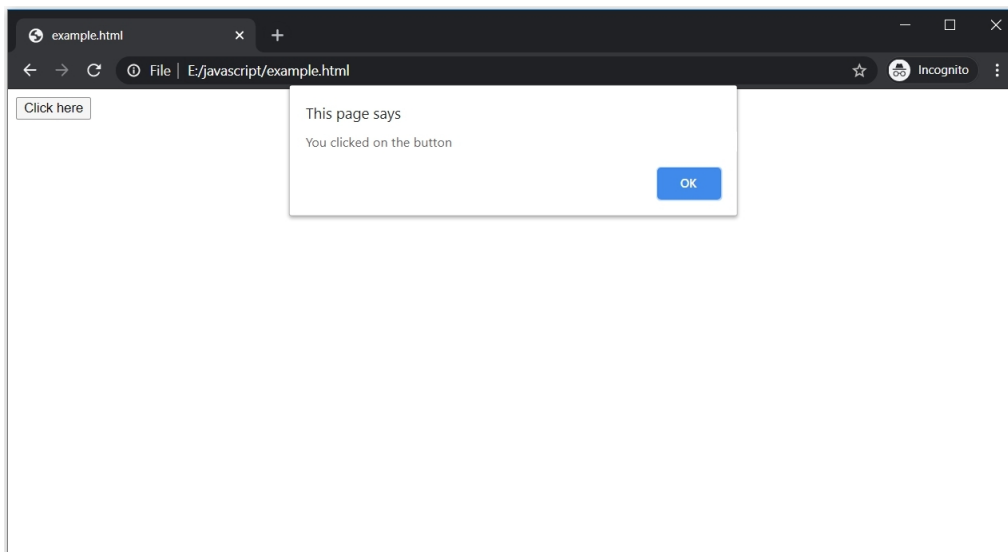
The onclick event is the most basic HTML event. As the name suggests, this **event triggers something when an element is clicked on**.

```

1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6          <button onclick="alert('You clicked on the button')">
7              click here
8          </button>
9      </body>
10 </html>

```

As mentioned earlier, HTML events are attributes. So the button above has an onclick event and its value is an alert() function. Remember, **the value of an HTML event is always written inside quotes**. The value will be triggered when the button is clicked.



The pop up appears when the button is clicked. So this example was to give you a basic understanding of how HTML events are used.

Triggering a function

In the real-time, events are used to trigger JavaScript functions.

Generally, complicated things such as DOM manipulation require some kind of coding logic and there can be multiple lines of code. So it is not possible to write all these lines in the HTML, and that is why HTML functions are used.

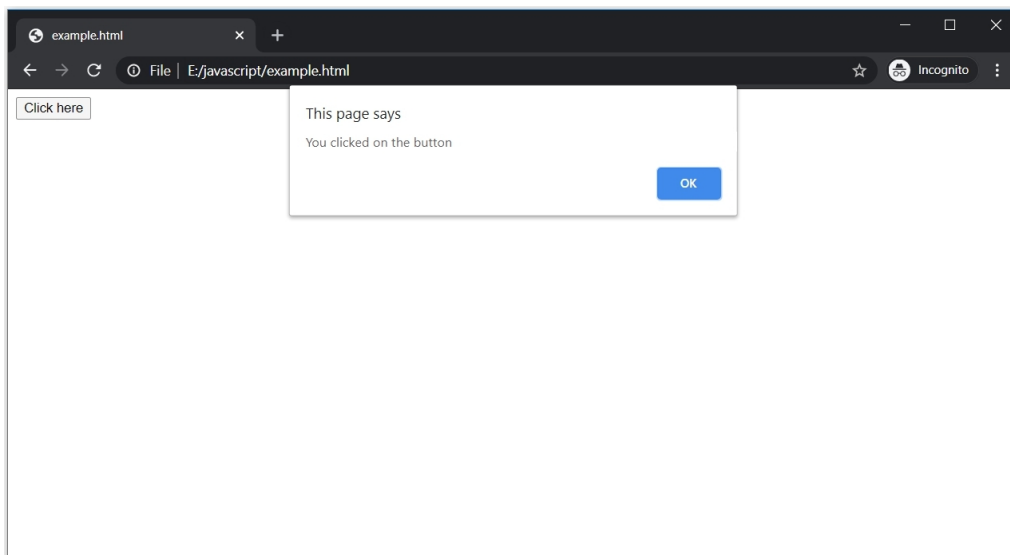
Let's put the alert() into a function.

```

1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6
7          <button onclick="message()">
8              Click here
9          </button>
10
11      </body>
12      <script>
13          function message(){
14
15              alert('You clicked on the button')
16
17          }
18      </script>
19  </html>

```

The function is placed inside the `<script>` tag , in fact, all the JavaScript is placed inside this tag only or in a separate JavaScript file. To trigger this function, we have to give it as the value of the onclick event.



This is how we trigger a function using HTML events. In the upcoming chapter, we will use the same concept to work with DOM.

Commonly used HTML events

Let's discuss some of the commonly used HTML events categorically.

Mouse events

- onclick - triggers on the single click of the mouse.
- ondblclick - triggers on the double click of the mouse.
- onmouseover - triggers when the mouse moves over an element.
- onwheel - triggers when the wheel of the mouse moves over an element.

Keyboard events

- onkeydown - triggers when a key is being pressed.
- onkeypress - triggers when a key is pressed.
- onkeyup - triggers when a key is released.

Window events

- onload - triggers when a window is completely loaded.
- onunload - triggers when a window is closed.
- onresize - triggers when a window is resized.

Form events

- onchange - triggers when the value of an element is changed.
- onsubmit - triggers when a form is submitted.
- onreset - triggers when a form is reset.

Drag events

- ondrag - triggers when an element is dragged.
- ondrop - triggers when an element that is being dragged is dropped.

Summary

- HTML events make something happens. They are used just like other attributes.
- Different types of HTML events are mouse, keyboard, drag, window, media, and form events.
- The value of an event is written inside the quotes. What will happens next depends on the value.
- Generally, functions are triggered using HTML events because they can have multiple lines of codes inside them.

Finding elements

Till now, we discussed JavaScript, DOM, and HTML events. These three concepts are necessary if you want to understand DOM manipulation properly.

In this chapter, we will discuss two very important concepts - **finding elements using various methods and the innerHTML property**.

innerHTML property

To manipulate any HTML element or its value, we first need to find it. We cannot do anything if we do not have the link of the HTML elements in the JavaScript of the HTML file. For this, there are few methods, that we will discuss next, but first need to understand the innerHTML property.

The innerHTML property is used to get the access to content of the HTML elements. The methods will only find the element, but its value is accessed using the innerHTML property only.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6          <p>
7              This is a paragraph.
8          </p>
9      </body>
10 </html>
```

With the methods, we can find the `<p>` tag, but the content, i.e. "This is a paragraph." is accessed using the innerHTML property.

Ways to find HTML elements

There are three ways of finding HTML elements

- `document.getElementById()`
- `document.getElementsByTagName()`
- `document.getElementsByClassName()`

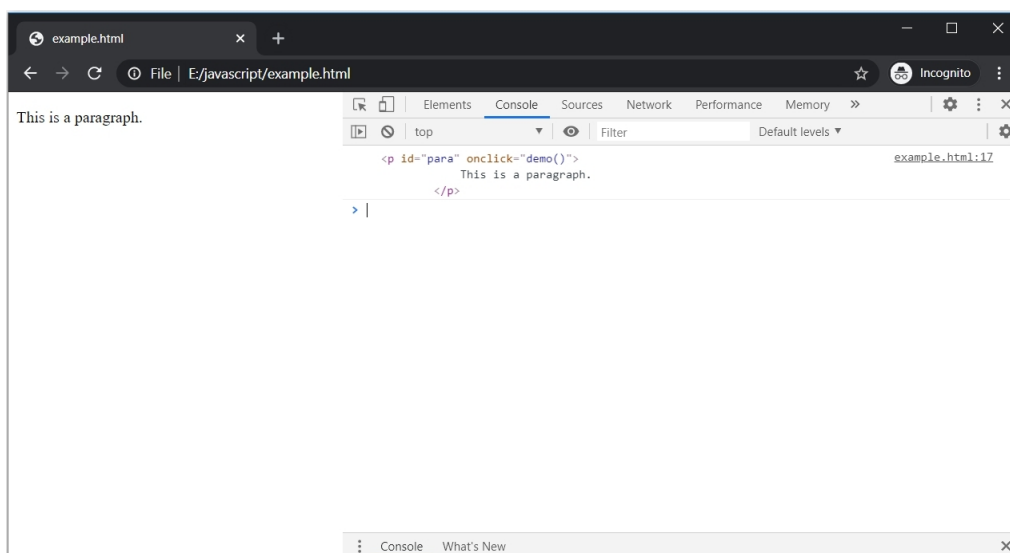
document.getElementById()

The document.getElementById() method is the most common way to find HTML elements.

The value passed to the document.getElementById() method is **the id of the element** that we want to find.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6
7          <p id = "para" onclick="demo()">
8              This is a paragraph.
9          </p>
10
11      </body>
12
13      <script>
14          function demo(){
15
16              var ele = document.getElementById("para");
17              console.log(ele)
18          }
19      </script>
20  </html>
```

In the HTML, the id of the <p> tag is passed to the method and then, the value is stored inside a variable named, ele. The console statement in the next line prints the value of ele. Let's check.

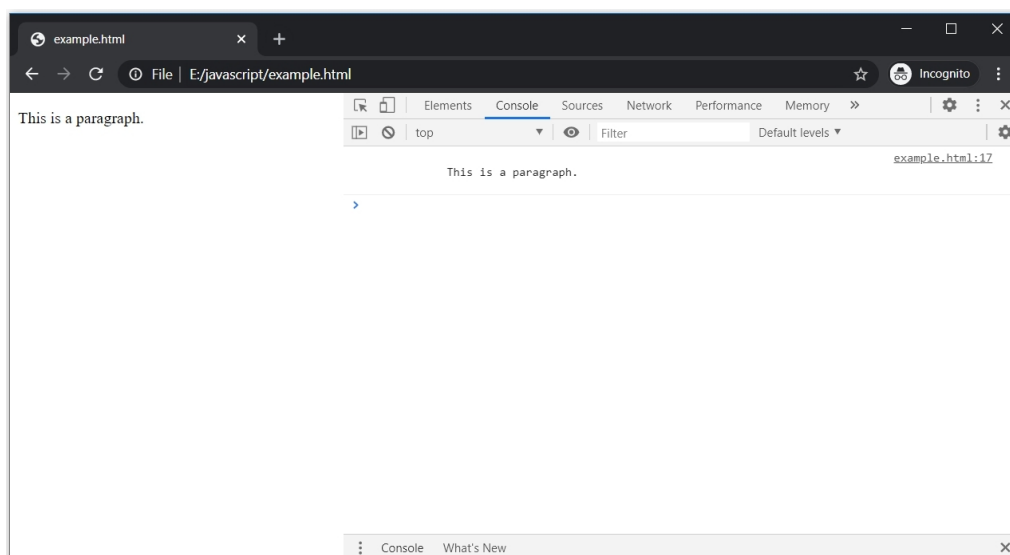


The value in the console appears after clicking the paragraph and it is nothing but, the `<p>` tag itself.

To access the content, we will use the `innerHTML` property.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4    </head>
5    <body>
6
7      <p id = "para" onclick="demo()">
8        |   This is a paragraph.
9      </p>
10
11    </body>
12
13    <script>
14      function demo(){
15
16        var ele = document.getElementById("para").innerHTML;
17        console.log(ele)
18      }
19    </script>
20  </html>
```

Let's again check the console.



This time, the value of the variable is equal to the content of the `<p>` tag.

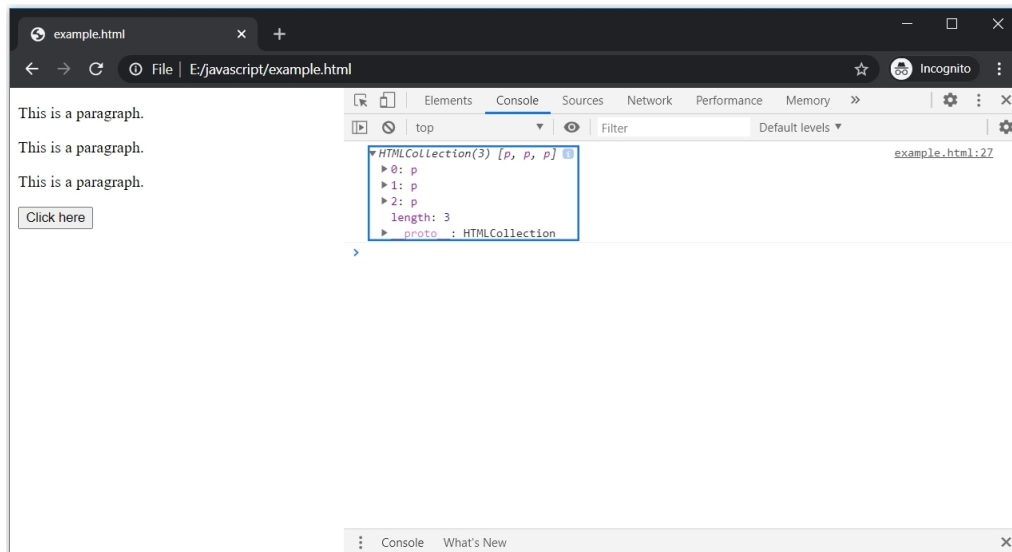
So, this is how a method and `innerHTML` are combined to get the content of an element.

document.getElementsByTagName()

The second way is to find elements **by the tag name**.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6
7          <p>
8              |   This is a paragraph.
9          </p>
10         <p>
11             |   This is a paragraph.
12         </p>
13         <p>
14             |   This is a paragraph.
15         </p>
16
17         <button onclick="demo()">
18             |   Click here
19         </button>
20
21     </body>
22
23     <script>
24         function demo(){
25
26             var ele = document.getElementsByTagName("p");
27             console.log(ele)
28         }
29     </script>
30 </html>
```

There are three <p> tags and document.getElementsByTagName() will find all of these. Let's check the console.



The `document.getElementsByTagName()` returns an array with all the elements. Similarly, the `innerHTML` property will also return an array with all the content.

`document.getElementsByClassName()`

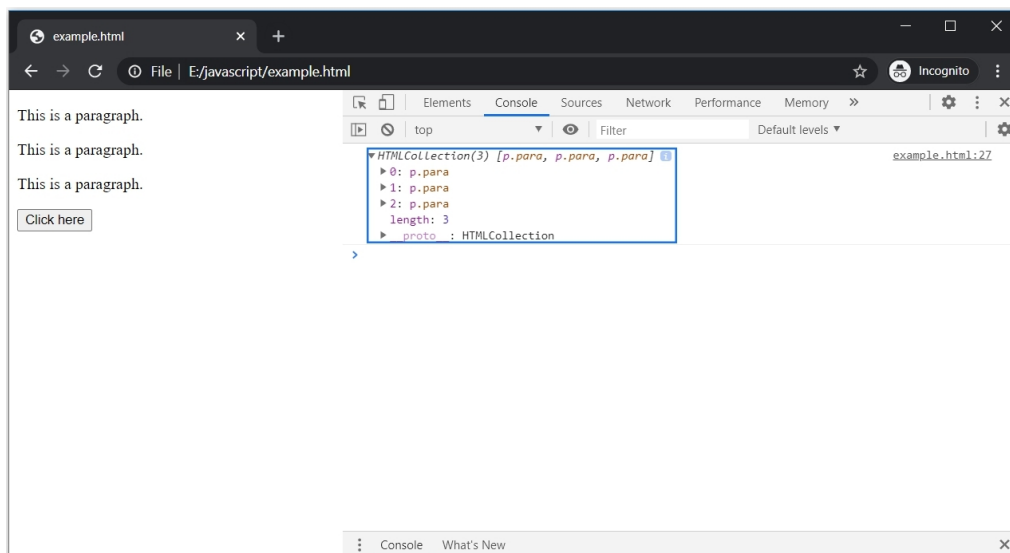
Similar to the `document.getElementsByTagName()` method, the `document.getElementsByClassName()` method is also used to find multiple elements. **It uses class name to find elements.**

```

1  <!DOCTYPE html>
2  <html>
3    <head>
4    </head>
5    <body>
6
7      <p class = "para">
8        |   This is a paragraph.
9      </p>
10     <p class = "para">
11       |   This is a paragraph.
12     </p>
13     <p class = "para">
14       |   This is a paragraph.
15     </p>
16
17     <button onclick="demo()">
18       |   Click here
19     </button>
20
21   </body>
22
23   <script>
24     function demo(){
25
26       var ele = document.getElementsByClassName("para");
27       console.log(ele)
28     }
29   </script>
30 </html>

```

It also returns an array.



Summary

- The innerHTML property is used to get the content of and element.

- There are three ways to find elements in HTML - `document.getElementById()`, `document.getElementsByTagName()`, and `document.getElementsByClassName()`.
- The `document.getElementById()` method finds an element using the id of that element.
- The `document.getElementsByTagName()` method find elements using the tag name.
- The `document.getElementsByClassName()` method find elements using the class of the elements.

Content and CSS with JavaScript

Let's move further and discuss, how can we change the content and CSS of HTML elements.

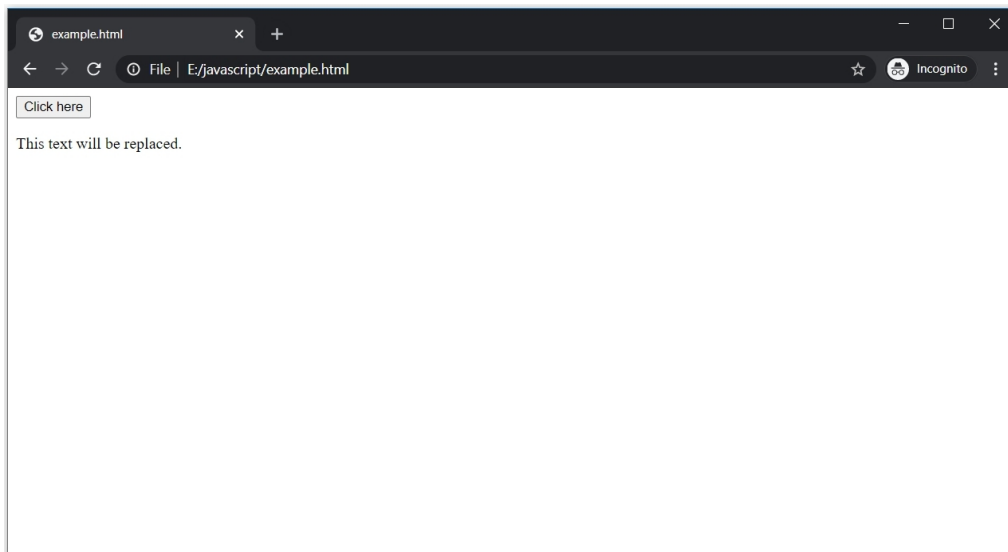
Changing content using innerHTML property

In the last chapter, we discussed how the methods like `document.getElementById()` can be combined with `innerHTML` property to access an element's content. The main usage of this property is to change the content.

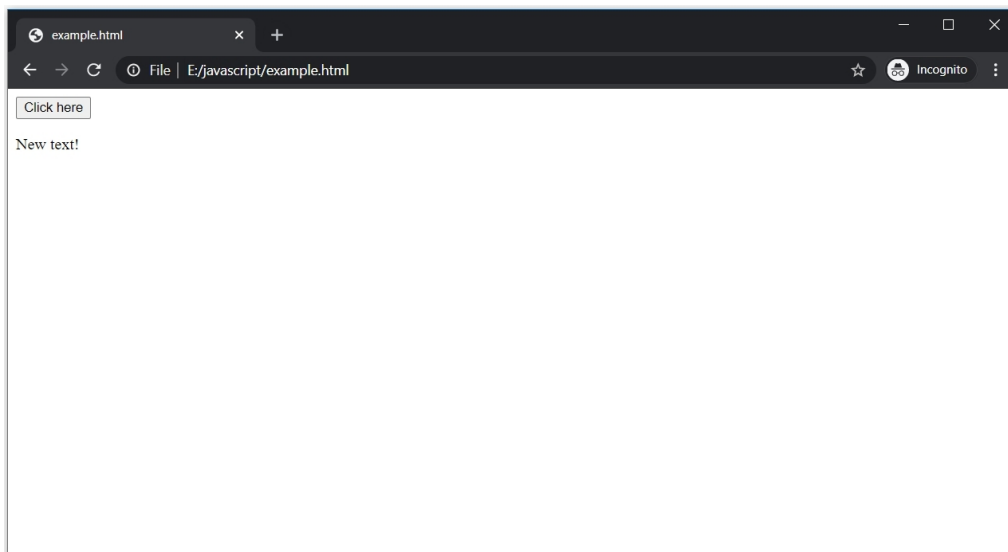
```
1  <!DOCTYPE html>
2  <html>
3    <head>
4    </head>
5    <body>
6
7      <button onclick="demo()">
8        Click here
9      </button>
10
11      <p id="para">
12        This text will be replaced.
13      </p>
14
15    </body>
16
17    <script>
18      function demo(){
19
20        var ele = document.getElementById("para");
21
22        ele.innerHTML = "New text!";
23
24      }
25    </script>
26  </html>
```

This is a dynamic HTML page. The page has a button and a paragraph. **Clicking on this button will replace the text of the paragraph.**

Before the button is clicked:



After the button is clicked:



Let's see what is happening here.

First, the `document.getElementById()` method is used to find the `<p>` tag. Its reference is stored in a variable named, `ele`. Then, this reference is used with `innerHTML` property to assign a new value to it. And everything is done, when the `demo()` function is triggered after the button is clicked.

The case in other two methods - `document.getElementsByTagName()` and `document.getElementsByClassName()` is different. The id is always unique in a HTML document, so `document.getElementById()` selects only a single element, but there can be multiple tags or multiple tags with same class name. So `document.getElementsByTagName()`

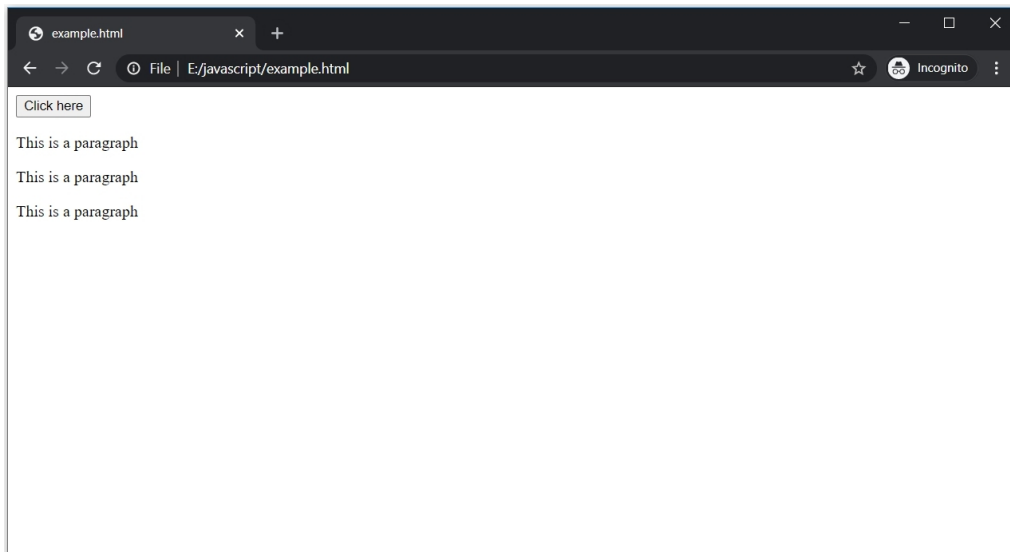
`document.getElementsByClassName()` stores the found elements in an array and we have to access them accordingly. Let's see how to do it.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6
7          <button onclick="demo()">
8              Click here
9          </button>
10
11          <p>This is a paragraph</p>
12          <p>This is a paragraph</p>
13          <p>This is a paragraph</p>
14
15      </body>
16
17      <script>
18          function demo(){
19
20              var ele = document.getElementsByTagName("p");
21
22              ele[0].innerHTML = "Text replaced!";
23
24          }
25      </script>
26  </html>
```

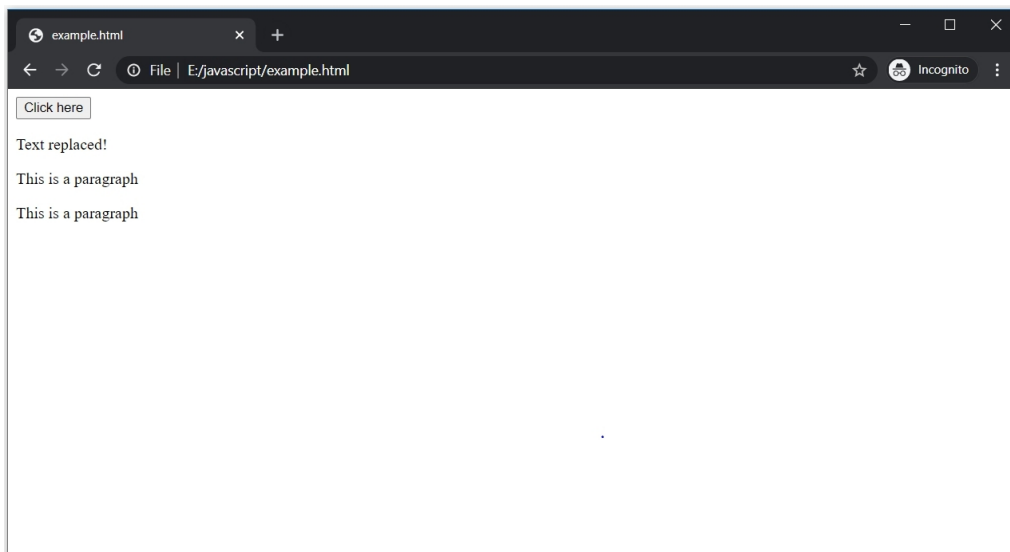
There are three `<p>` tags. In `demo()` function, `document.getElementsByTagName()` method is used to find them. It will end up finding all the `<p>` tags. We can't access them like we did in the case of `document.getElementById()` method. We have to access them using the index.

The first `<p>` is on the 0th index, the second is on the 1st index, and goes on. In the above example, the text of the first `<p>` tag is replaced by using the `innerHTML` property on the `<p>` tag that is on the 0th index.

Before the button is clicked:



After the button is clicked:



Similarly, other `<p>` tags can be accessed using their respective indexes.

Changing values of the attributes

It is also possible to change the value of an attribute with JavaScript. **There is no special property to change an attribute's value.** We can simply use the property name after finding the element and assign it a new value.

Suppose, there an image on the page and When clicked on it, a new image replaces the old one. To do this, we need to assign a new value to the `src` attribute of the `<img>` tag.

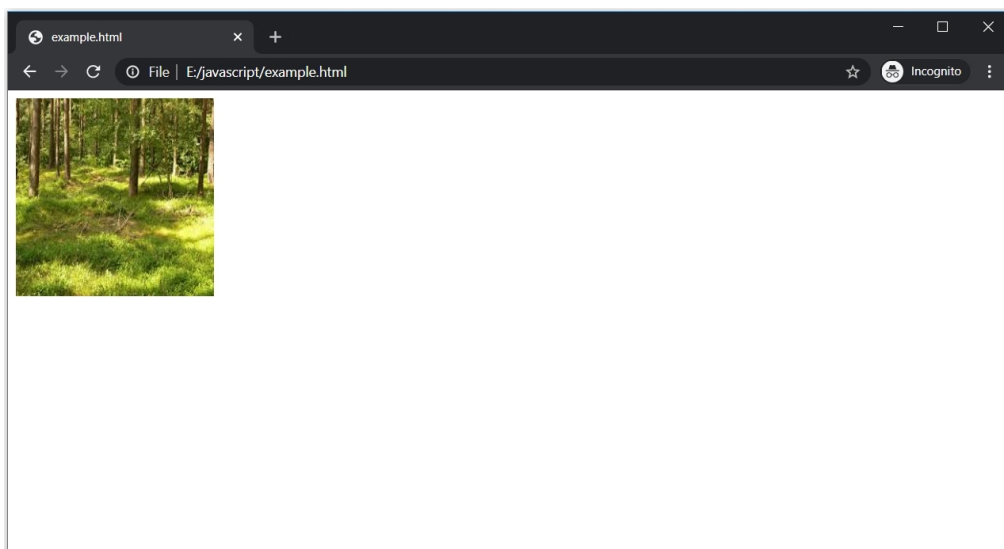
```

1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6          
13
14
15      </body>
16
17      <script>
18          function demo(){
19
20              var ele = document.getElementById("image");
21              ele.src = "./images/mountain.jpg";
22
23          }
24      </script>
25  </html>

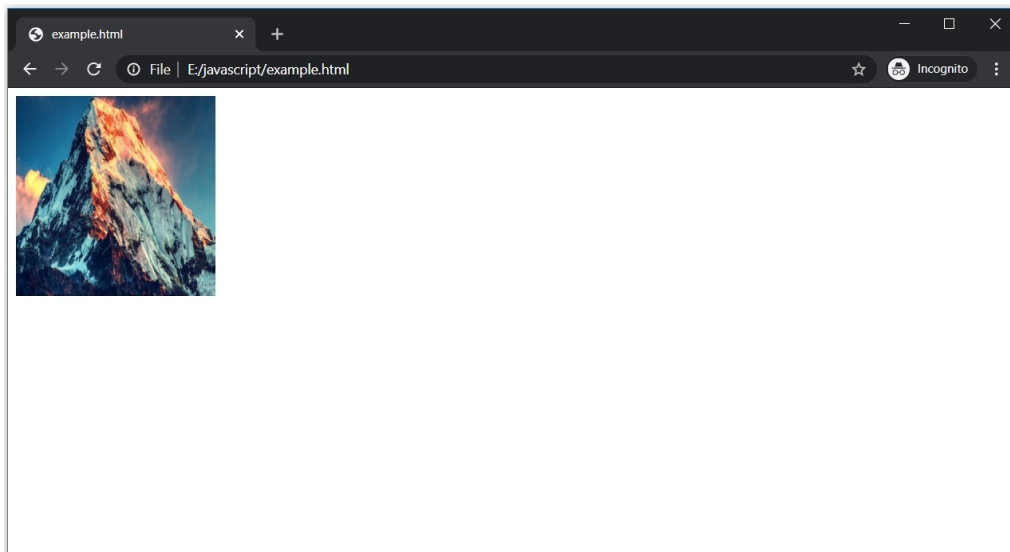
```

Here, the onclick() event is used on the tag. In the demo() function, first, the tag is located using the document.getElementById() method and then, the src attribute is assigned a new value.

Before clicking on the image:



After clicking on the image:



Similarly, we can change the value of any attribute.

Changing CSS

Not only the content and attribute values, but we can also change the CSS with JavaScript.

First, we have to access the style attribute of the element and then, apply the required CSS property to it with the value.

Suppose, there is a paragraph whose color is red, and when clicked on it, the color changes to blue.

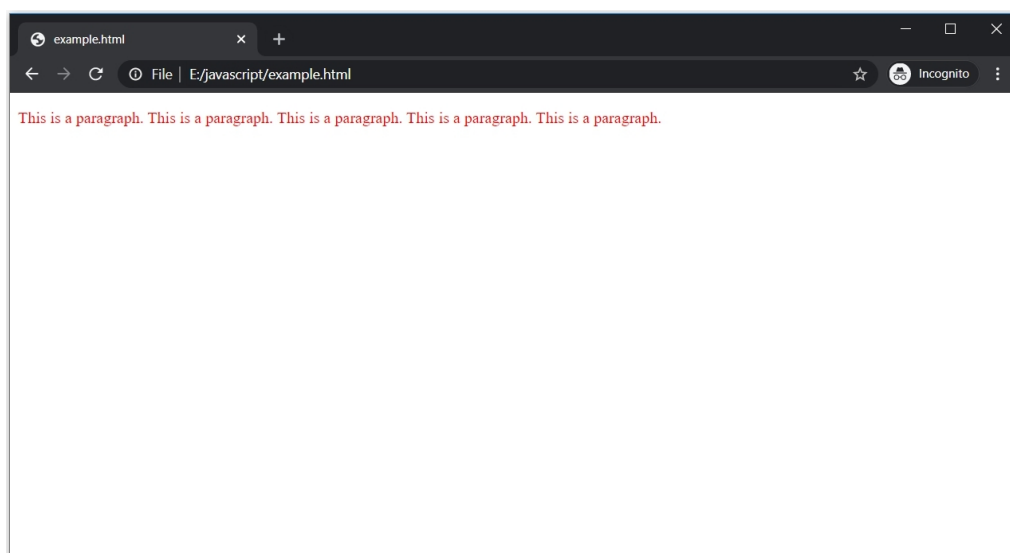
```

1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6
7          <p id="para" style="color : ■ red" onclick="demo()">
8              This is a paragraph.
9              This is a paragraph.
10             This is a paragraph.
11             This is a paragraph.
12             This is a paragraph.
13         </p>
14
15     </body>
16
17     <script>
18         function demo(){
19
20             var ele = document.getElementById("para");
21
22             ele.style.color = "blue";
23
24         }
25     </script>
26 </html>

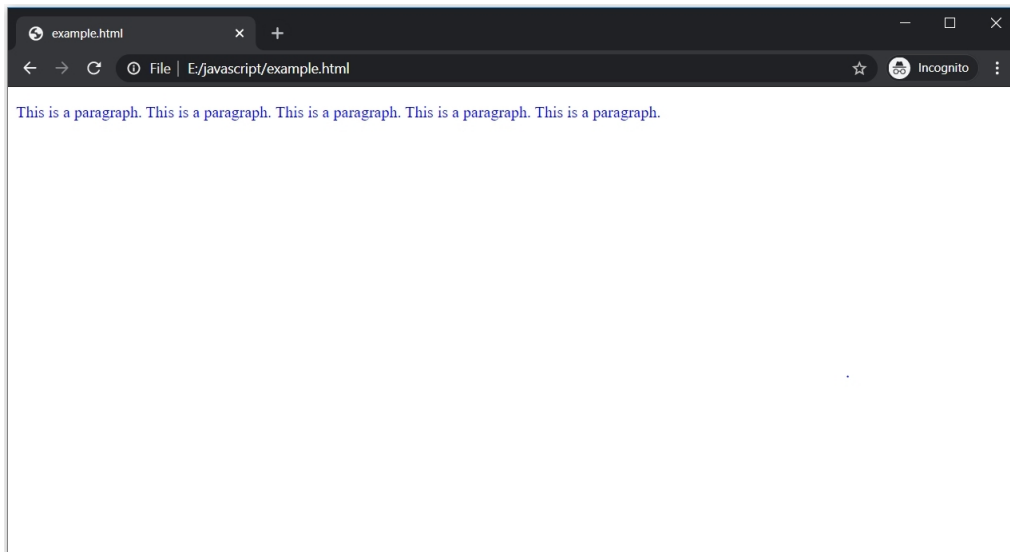
```

Here, the style attribute is accessed and then the color property is applied to it with a new value.

Before clicking on the paragraph:



After clicking on the paragraph:



Summary

- The `innerHTML` property is used to change the content of an HTML element.
- To change the value of an attribute, use the attribute name, and assign the new value.
- The CSS can be changed by using the `style` attribute and then applying the property with the new value.

Creating and removing elements

Every element in an HTML document is represented as a Node in the DOM tree. In the last chapter, we discussed how to update the content of an existing element. But with JavaScript, we can also add new elements and remove the existing one.

There are a few methods that are used to create or remove elements. In this chapter, we will discuss how to use these methods in JavaScript.

document.createElement()

As the name suggests, the `document.createElement()` method is **used to create an HTML element**.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4    </head>
5    <body>
6      <div id="main">
7        <button onclick="demo()">Click here!</button>
8      </div>
9    </body>
10
11    <script>
12      function demo(){
13
14        var paragraph = document.createElement("p");
15
16        var text = document.createTextNode("This is a paragraph");
17
18      }
19    </script>
20  </html>
```

The `document.createElement()` method is used to create a `<p>` element. The element name should be passed to this method. Now, we have a `<p>` tag but there is no content in it yet. The next step is to use the `document.createTextNode()` method to create a text node.

As of now, we have a new element and the content that is to be added to it. But this newly created element is not added in the HTML document. It is not present in the DOM tree.

To add this to the DOM, We have a separate method.

appendChild()

In the last example, we create two nodes - `<p>` tag and the text. We have to append the `<p>` to the HTML document, but first, we need to append the text node to the newly create `<p>` tag.

The `appendChild()` method appends a new node as the last child node. So let's append the text node to the `<p>` tag and then, we will append this whole element as one of the last child nodes in the HTML document

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6          <div id="main">
7              <button onclick="demo()">Click here!</button>
8          </div>
9      </body>
10
11      <script>
12          function demo(){
13              var paragraph = document.createElement("p");
14              var text = document.createTextNode("This is a paragraph");
15              paragraph.appendChild(text);
16          }
17      </script>
18  </html>
```

Now, we have a complete `<p>` tag. But still, it is not present in the HTML document.

The `<body>` tag has `<div>` as a node, and in turn, the `<div>` tag has a `<button>` as its node. Now, we will append this newly created `<p>` tag as a child node of the `<div>` tag using the `appendChild()` method. This means, we have to invoke the `appendChild()` method on the `<div>` tag and for this, first we need to find this `<div>` tag.

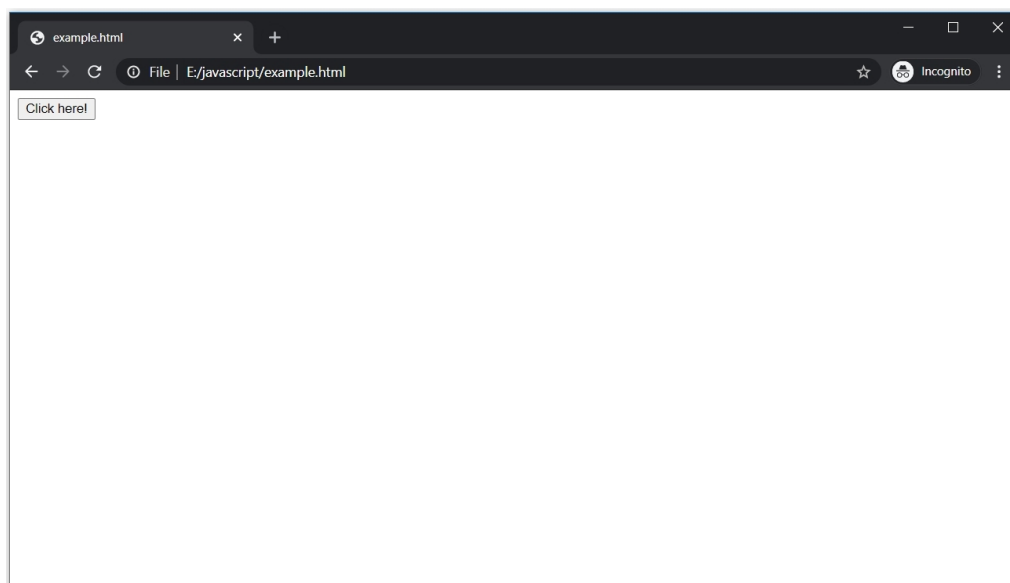

```

1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6          <div id="main">
7              <button onclick="demo()">Click here!</button>
8          </div>
9      </body>
10
11
12
13      <script>
14          function demo(){
15
16              var paragraph = document.createElement("p");
17
18              var text = document.createTextNode("This is a paragraph");
19
20              paragraph.appendChild(text);
21
22              var div = document.getElementById("main");
23              div.appendChild(paragraph)
24          }
25      </script>
26  </html>

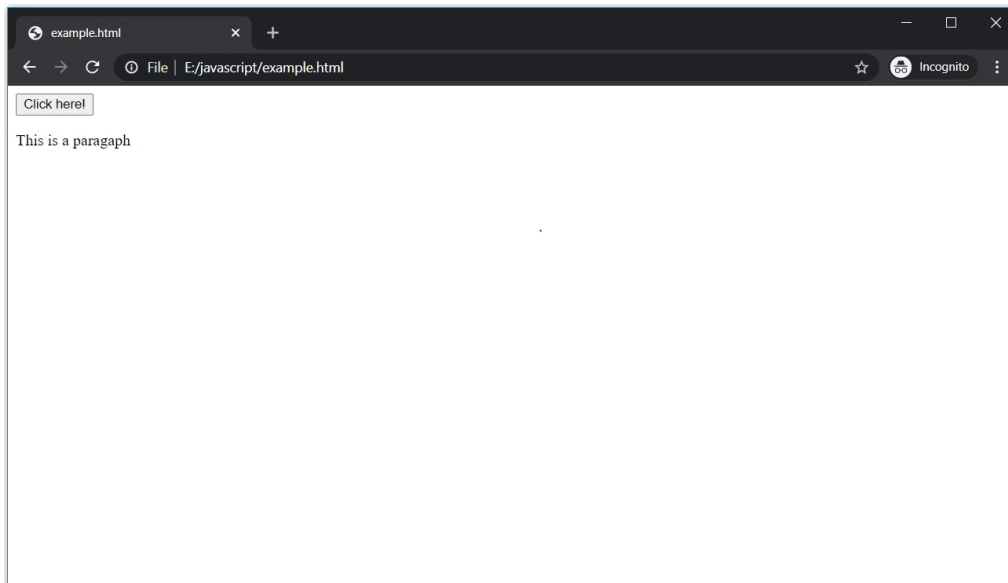
```

First, the `<div>` tag is located using the `document.getElementById()` method and then, the new node is appended to it using the `appendChild()` method.

Before the button is clicked:



After the button is clicked:



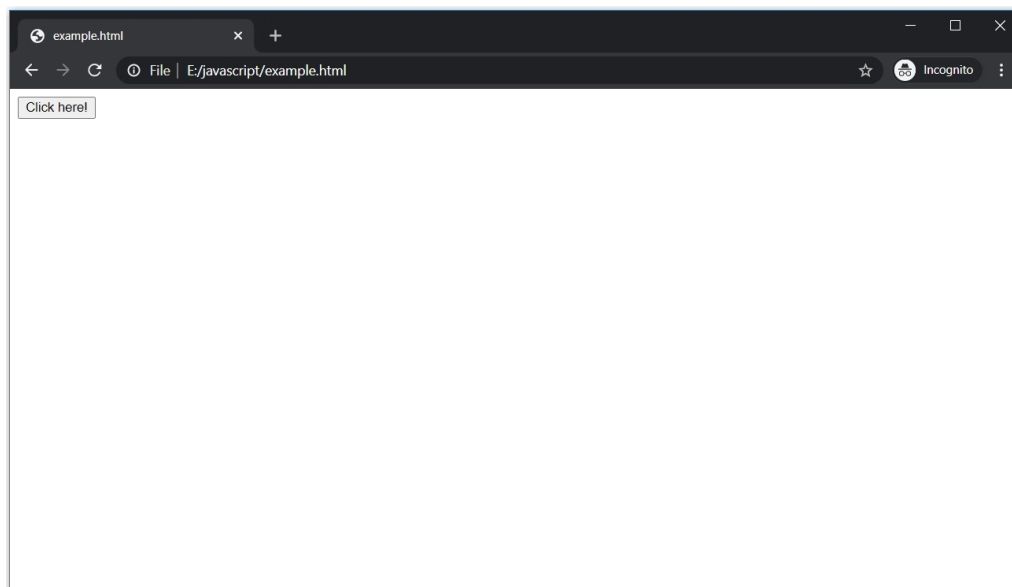
Let's discuss another example, this time using the innerHTML property.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4    </head>
5    <body>
6      <div id="main">
7        <button onclick="demo()">Click here!</button>
8      </div>
9    </body>
10
11  </body>
12
13  <script>
14    function demo(){
15      var button = document.createElement("button");
16      button.innerHTML = "New button"
17    }
18  </script>
19
20  </html>
```

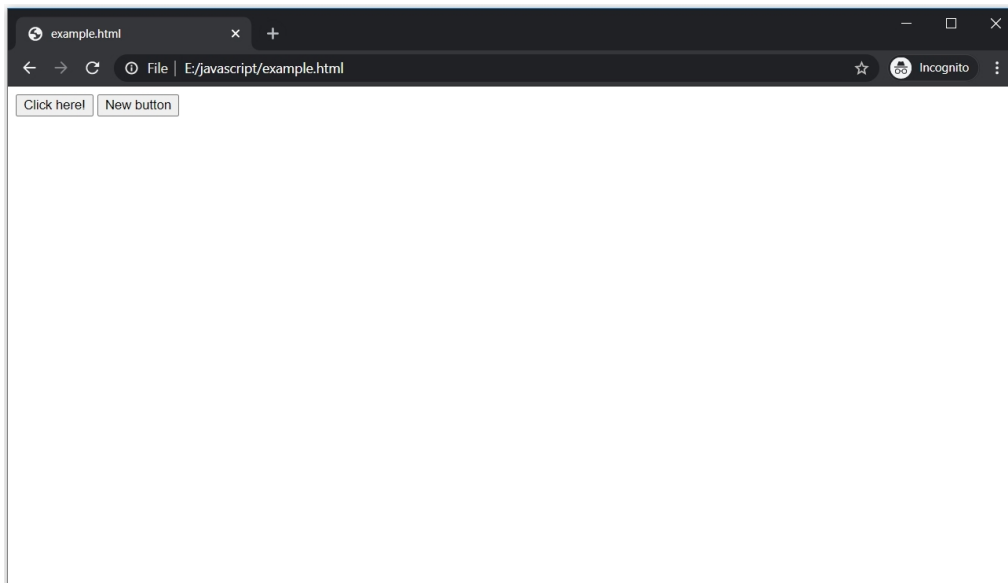
In the example, a button is created using the `document.createElement()` method and then, `innerHTML` property is used on it to assign a value. Let's append this newly created element to the `<div>` tag.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4    </head>
5    <body>
6      <div id="main">
7        <button onclick="demo()">Click here!</button>
8      </div>
9    </body>
10
11    <script>
12      function demo(){
13
14        var button = document.createElement("button");
15
16        button.innerHTML = "New button"
17
18        var div = document.getElementById("main");
19
20        div.appendChild(button)
21
22      }
23    </script>
24  </html>
```

Before the button is clicked:



After the button is clicked:



insertBefore()

The insertBefore() method is used to insert an element right before an existing element.

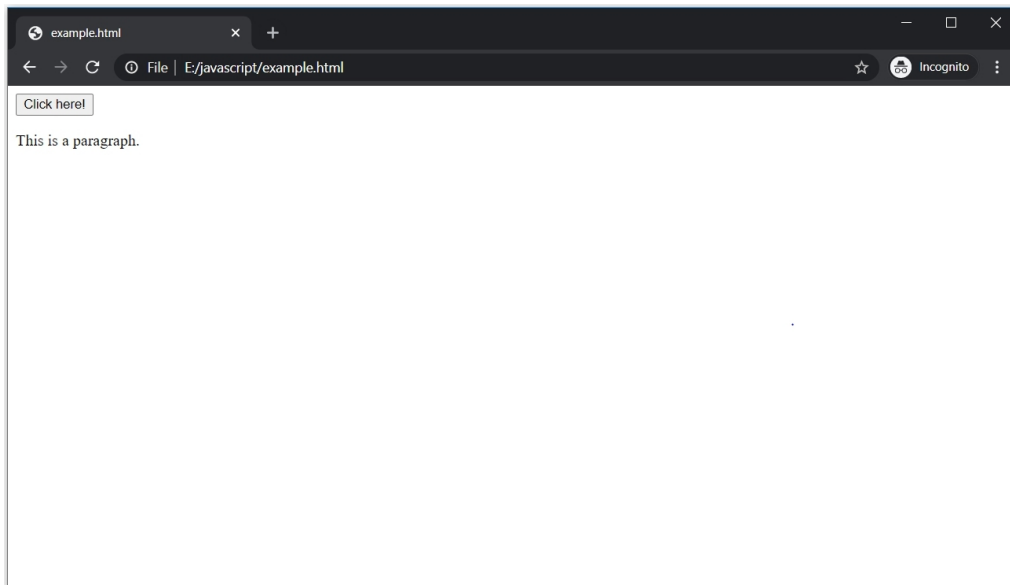
Suppose, we have a `<p>` tag and we want to add a `<h1>` tag before that `<p>` tag.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6          <div id="main">
7              <button onclick="demo()">Click here!</button>
8              <p id="para"> This is a paragraph.</p>
9          </div>
10
11      </body>
12
13      <script>
14          function demo(){
15
16              var heading = document.createElement("h1");
17              var text = document.createTextNode("Heading");
18
19              heading.appendChild(text);
20
21              var div = document.getElementById("main");
22              var paragraph = document.getElementById("para");
23
24              div.insertBefore(heading, paragraph)
25          }
26      </script>
27
28  </html>
```

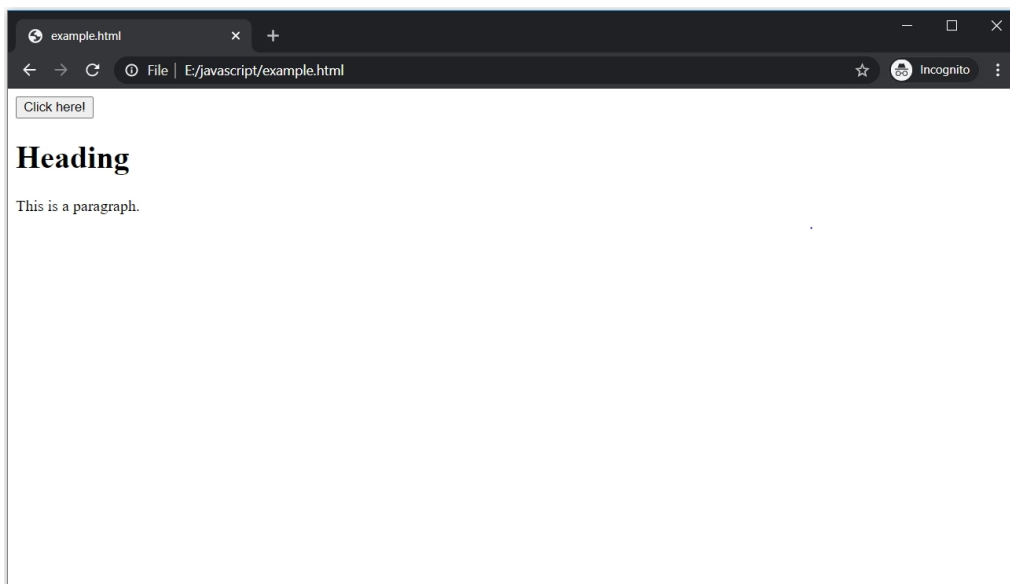
First, a new `<h1>` element is created. Now, we need to place this newly created element right before the `<p>` tag, but before it, we need to locate the `<div>` tag because the `<p>` tag exists in it.

The `insertBefore()` method is invoked using the div and, **both the elements are passed to this method.**

Before the button is clicked:



After the button is clicked:



`document.createAttribute()`

We can also create a new attribute for an existing element as well as for a newly created element.

Suppose, there is a paragraph and on the button click, a class attribute with CSS is applied to it.

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <style>
5              .css {
6                  font-size: 50px;
7                  color : blue;
8              }
9          </style>
10     </head>
11     <body>
12
13         <button onclick="demo()">Click here!</button>
14         <p id="para"> This is a paragraph.</p>
15
16     </body>
17     <script>
18         function demo(){
19
20             var paragraph = document.getElementById("para")
21
22             var attribute = document.createAttribute("class");
23             attribute.value = "css";
24
25         }
26     </script>
27 </html>
```

First, the `<p>` tag is located using the `document.getElementById()` method. Next, we create a class attribute using the `document.createAttribute()` method, and then assigned a value to it. The class is already defined in the head section.

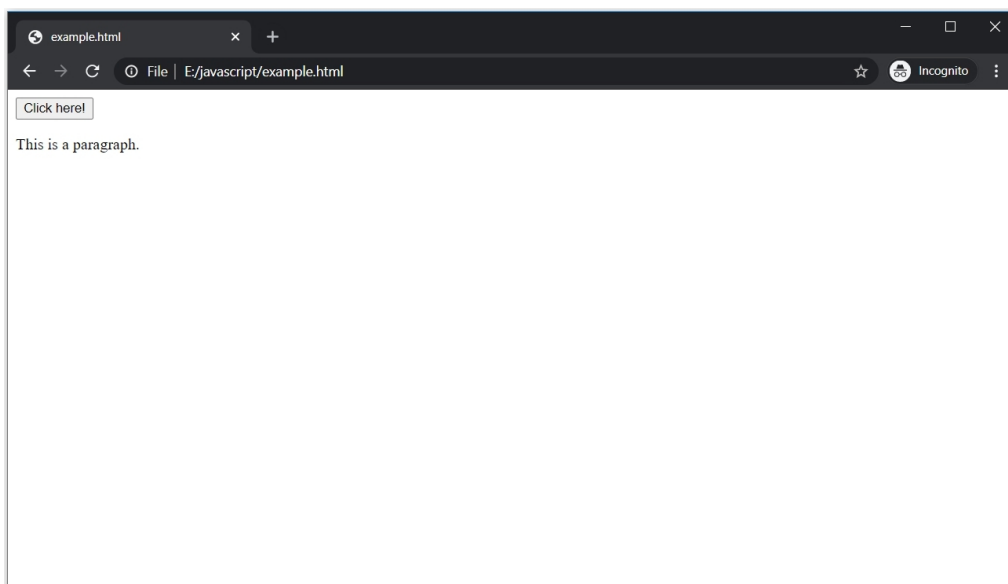
As of now, we have created an attribute but it is not added to the `<p>` tag. To do this, **we need to use the `setAttributeNode()` method.**

```

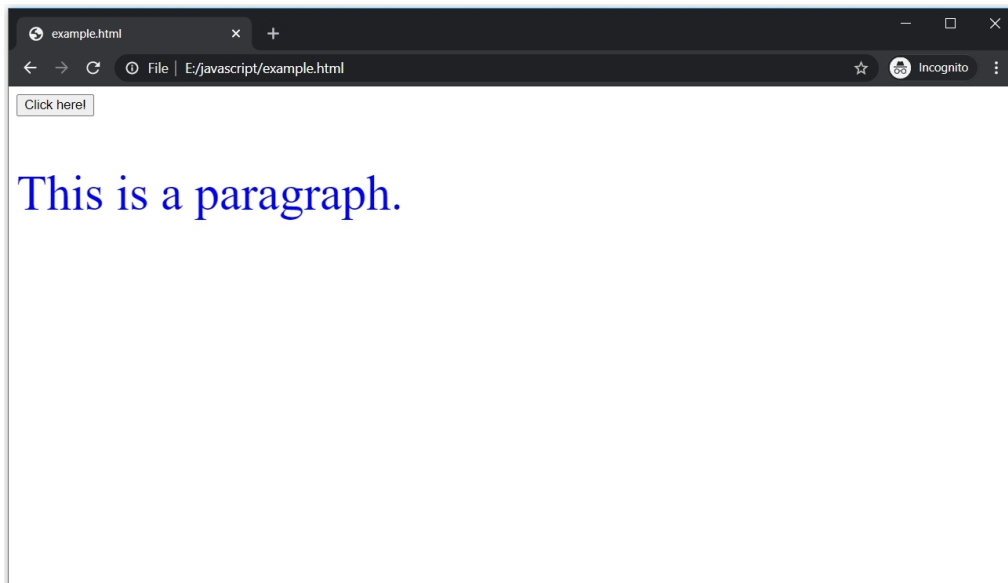
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <style>
5              .css {
6                  font-size: 50px;
7                  color : blue;
8              }
9          </style>
10     </head>
11     <body>
12
13         <button onclick="demo()">Click here!</button>
14         <p id="para"> This is a paragraph.</p>
15
16     </body>
17     <script>
18         function demo(){
19
20             var paragraph = document.getElementById("para")
21
22             var attribute = document.createAttribute("class");
23             attribute.value = "css";
24
25             paragraph.setAttributeNode(attribute);
26         }
27     </script>
28 </html>

```

Before the button is clicked:



After the button is clicked:



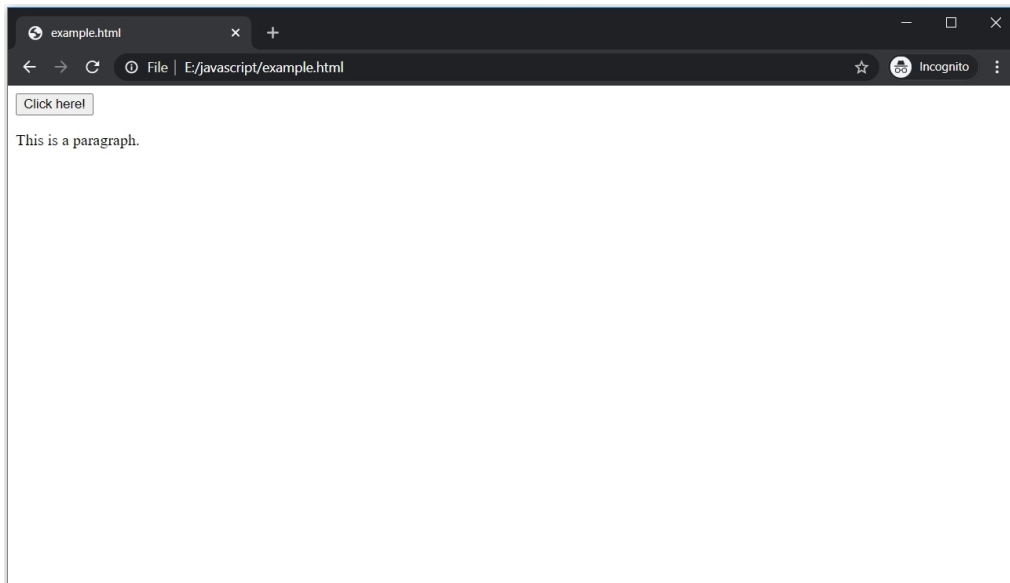
remove()

The `remove()` method is used to remove an element from an HTML document.

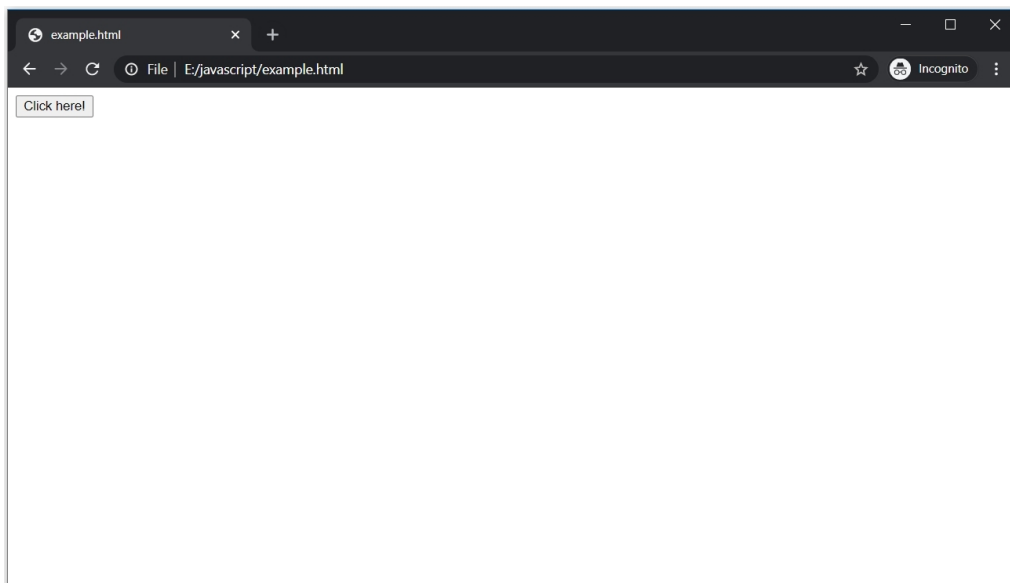
```
1  <!DOCTYPE html>
2  <html>
3      <head>
4      </head>
5      <body>
6
7          <button onclick="demo()">Click here!</button>
8          <p id="para"> This is a paragraph.</p>
9
10     </body>
11
12     <script>
13         function demo(){
14
15             var paragraph = document.getElementById("para");
16
17             paragraph.remove();
18
19         }
20     </script>
21 </html>
```

The `<p>` tag is located using the `document.getElementById()` method and then, it is used to invoke the `remove()` method.

Before the button is clicked:



After the button is clicked:



Summary

- The `document.createElement()` method is used to create a new element.
- To create a text node, use `document.createTextNode()` method and to assign a value to an element, use the `innerHTML` property with `document.createElement()` method.
- The `appendChild()` method is used to append a new element as the last child node of an existing node.
- The `insertBefore()` method is used to insert a new element right before an existing element.

- A new attribute can be created using the `document.createAttribute()` method while its value can be assigned using the `setAttributeNode()` method.
- To remove an existing element, use the `remove()` method.